Introduction to Machine Learning M146 | UCLA Spring Quarter 2026
Prof. Vwani Roychowdhury | 24th April 2026

TAs: Kartik Sharma, M. Yigit Turali

Homework #2

Due: 8th May 2026, Friday, before 11:59 pm

Problem 1 (Replacing Hinge Loss with Logistic Loss)

In standard soft-margin SVM, the hinge-loss objective is

$$J_{\text{hinge}}(w, b) = \sum_{n=1}^{N} \max(0,\ 1 - y_n(w^T x_n + b)) + \frac{\lambda}{2}\|w\|^2,$$

where $y_n \in \{-1, +1\}$ and $\lambda > 0$.

In this problem, we replace the hinge loss by the logistic loss and study how the training behavior changes. Consider the objective:

$$J_{\log}(w, b) = \sum_{n=1}^{N} \log\big(1 + e^{-y_n(w^T x_n + b)}\big) + \frac{\lambda}{2}\|w\|^2.$$

Define the margin of the n-th example as

$$m_n = y_n(w^T x_n + b).$$

(a) **Gradient for one example.**

For a single training example (x, y), define

$$\ell_{\log}(w, b) = \log\big(1 + e^{-y(w^T x + b)}\big).$$

i. Compute $\nabla_w \ell_{\log}(w, b)$.

ii. Compute $\frac{\partial \ell_{\log}(w,b)}{\partial b}$.

(b) **Gradient of the full objective.**

Compute $\nabla_w J_{\log}(w, b)$ and $\frac{\partial J_{\log}(w,b)}{\partial b}$.

(c) **Effect of the margin.**

Using your expression from part (1), explain how the magnitude of the gradient contribution of one example depends on its margin

$$m = y(w^T x + b).$$

In particular, discuss the following three cases qualitatively:

i. $m \gg 0$ (correctly classified with large positive margin)

ii. $m \approx 0$ (close to the decision boundary)

iii. $m \ll 0$ (confidently misclassified)

(d) **Comparison with hinge loss.**

Recall that the hinge loss for one example is

$$\ell_{\text{hinge}}(w, b) = \max\big(0,\ 1 - y(w^T x + b)\big).$$

Compare logistic loss and hinge loss in terms of which training examples affect the update:

i. Under hinge loss, when does a training example contribute zero gradient (or subgradient)?

ii. Under logistic loss, can a training example ever contribute exactly zero gradient?

iii. Based on this, which loss continues to be influenced by correctly classified points with very large positive margin?

(e) **Sensitivity to hard examples.**

Suppose one training example is very far on the wrong side of the decision boundary, so that

$$y(w^T x + b) \ll 0.$$

Using the form of the gradient, explain whether logistic loss reacts more strongly, less strongly, or similarly to hinge loss on such an example.

(f) **Decision boundary.**

The classifier obtained after training still predicts using

$$h(x) = \text{sign}(w^T x + b).$$

Does replacing hinge loss by logistic loss change the form of the decision boundary, or only the training objective? Briefly explain.

Problem 2 (Kernel Geometry and Functional Behavior)

Consider the Gaussian (RBF) kernel

$$k(x, z) = \exp\Big(-\|x - z\|^2\Big), \quad x, z \in \mathbb{R}^d.$$

Recall that a kernel corresponds to an (implicit) feature map $\phi(x)$ such that

$$k(x, z) = \phi(x)^T \phi(z).$$

(a) **Algebraic structure.**

Rewrite $k(x, z)$ in terms of $x^T x$, $z^T z$, and $x^T z$.

(b) **Kernelized distance.**

Define the squared distance in feature space as

$$d_k(x, z) = \|\phi(x) - \phi(z)\|^2.$$

i. Show that

$$d_k(x, z) = k(x, x) + k(z, z) - 2k(x, z).$$

ii. Compute $k(x, x)$ for the RBF kernel.

iii. Simplify $d_k(x, z)$ in terms of $k(x, z)$.

(c) **Geometric behavior.**

Using your expression from part (2), describe how $d_k(x, z)$ behaves in the following cases:

i. $\|x - z\| \to 0$

ii. $\|x - z\| \to \infty$

What does this imply about distances in the feature space induced by the RBF kernel?

(d) **Kernelized prediction.**

Consider a classifier of the form

$$h(x) = \sum_{n=1}^{N} \alpha_n k(x_n, x).$$

Explain what happens to $h(x)$ when the test point x is far from all training points $\{x_n\}_{n=1}^N$.

(e) **Implicit feature space.**

Based on the above observations, explain why it is not necessary to explicitly compute $\phi(x)$ for the RBF kernel, even if the corresponding feature space may be very high-dimensional.

Problem 3 (When a Function is Not a Kernel)

Consider the function

$$k(x, z) = \|x - z\|^2, \quad x, z \in \mathbb{R}^d.$$

(a) **Kernelized distance.**

Assume for the sake of argument that $k(x, z)$ is a valid kernel with feature map $\phi(x)$, so that

$$k(x, z) = \phi(x)^T \phi(z).$$

Define the squared distance in feature space:

$$d_k(x, z) = \|\phi(x) - \phi(z)\|^2.$$

i. Show that

$$d_k(x, z) = k(x, x) + k(z, z) - 2k(x, z).$$

ii. Compute $k(x, x)$.

iii. Simplify $d_k(x, z)$.

(b) **Contradiction.**

Using your result from part (1), explain why $d_k(x, z)$ cannot represent a valid squared Euclidean distance.

(c) **Gram matrix test.**

Consider two distinct points x_1, x_2, and form the Gram matrix

$$K = \begin{pmatrix} k(x_1, x_1) & k(x_1, x_2) \\ k(x_2, x_1) & k(x_2, x_2) \end{pmatrix}.$$

i. Compute K.

ii. Find its eigenvalues.

iii. Is K positive semidefinite?

(d) **Conclusion.**

Based on the above, explain why $k(x, z) = \|x - z\|^2$ is not a valid kernel function.

Problem 4 (Kernelized SVM and Decision-Boundary Geometry)

Imagine you are a data scientist embedded in a cardiology clinic. A physician hands you measurements from two cheap bedside instruments—the patient's *maximum heart rate achieved during a stress test* and the *ST depression induced by exercise relative to rest*—and asks: "can you tell me whether this patient has heart disease?" Two features, one binary label. This is the perfect setting to *see* how kernel choice reshapes the decision rule, and to connect what you proved on paper in Problems 1–2 of this homework to what the classifier actually does.

A note on tools. This problem is about *visualizing* what different kernels do geometrically, not about implementing SVMs from scratch—that comes in Problem 5. You may therefore use `sklearn.svm.SVC` and `sklearn.linear_model.LogisticRegression` here. You should still use `pandas` for data wrangling and `matplotlib`/`seaborn` for plotting.

Dataset. Download the *Heart Disease UCI* dataset from Kaggle:

https://www.kaggle.com/datasets/ronitf/heart-disease-uci

The original UCI dataset contains 13 clinical features and a binary target (`target` = 1 means presence of heart disease). For visualization, we will restrict attention to two continuous features throughout this problem:

Feature	Name	Meaning
x_1	`thalach`	Maximum heart rate achieved (bpm)
x_2	`oldpeak`	ST depression induced by exercise (mm)
y	`target`	1 if heart disease, 0 otherwise

(a) **Exploratory setup.**

(i) **Load and scatter.** [**4 pts**] Load the dataset, keep only the two features `thalach` and `oldpeak` together with `target`, and produce a 2D scatter plot of the full dataset coloured by label. Is the data linearly separable by eye? Are there regions where the two classes visibly overlap?
Why: Looking at the data first lets you build an expectation for what each kernel *should* be able to do before you fit anything. If the two classes already separate with a straight line, fancier kernels are unlikely to help.

(ii) **Train / test split and standardization.** [**4 pts**] Split 80/20 using `np.random.seed(42)`, and standardize both features to zero mean and unit variance using the **training set only**. Apply the same μ, σ to the test set. Why would it be incorrect to compute μ, σ on the combined train+test data? (See Problem 4(b)(iii) of Homework 1 if you need a refresher on data leakage.)

(b) **Linear SVM baseline.**

Fit a linear soft-margin SVM using `SVC(kernel='linear', C=1.0)` on the standardized training data.

(i) [**4 pts**] Plot the decision boundary $w^T x + b = 0$ together with the two margin lines $w^T x + b = \pm 1$ overlaid on a scatter of the training points. Highlight the support vectors (available via `svc.support_vectors_`).
Why: The support vectors are precisely the training examples whose margin satisfies $m_n \leq 1$—the same set of points that contribute non-zero subgradient in Problem 1(4a) of this homework. Seeing them as specific dots on your plot makes the theory tangible.

(ii) [**2 pts**] Report training and test accuracy.

(c) **Polynomial kernels.**

(i) [**4 pts**] Fit `SVC(kernel='poly', degree=d, C=1.0)` for $d \in \{2, 3, 5\}$. For each, report training and test accuracy.

(ii) [**4 pts**] Produce a 1×3 subplot grid of decision-boundary plots (one per degree). To draw a boundary for a non-linear kernel, evaluate the trained classifier on a fine grid of points covering the plot region and use `matplotlib`'s `contour` or `contourf` at level 0. Describe in 2–3 sentences how the boundary changes with degree. At what degree does overfitting start to look visually obvious (e.g. a boundary that makes unphysical excursions to chase individual training points)?

(d) **RBF (Gaussian) kernel.**

(i) [**4 pts**] Fit `SVC(kernel='rbf', gamma=`γ`, C=1.0)` for $\gamma \in \{0.1, 1, 10, 100\}$. `sklearn` parameterises the RBF kernel as $k(x, z) = \exp(-\gamma \|x - z\|^2)$, so larger γ means *faster* decay with distance. For each γ, report training and test accuracy.

(ii) [**6 pts**] Produce a 2×2 subplot grid of decision-boundary plots. Connect the visual behaviour to Problem 2(3) of this homework: as γ increases, $d_k(x, z) = 2(1 - e^{-\gamma \|x-z\|^2})$ saturates at 2 even for moderately separated points, so the classifier becomes more and more *local*—the boundary at large γ looks like a collection of small islands around training points. At what γ is the test accuracy best? At what γ has the model clearly overfit?

What to expect: $\gamma = 0.1$ should look close to linear (the kernel barely decays and every training point influences every test point); $\gamma = 100$ should look like a patchwork of tiny decision regions (the kernel decays so fast that each training point influences only a tiny neighbourhood). The sweet spot is in between.

(e) **Linking back to the theory.**

(i) **Loss does not change the boundary shape. [4 pts]** In Problem 1(6) of this homework you argued that replacing hinge loss with logistic loss does not change the *form* of the decision boundary—both are linear in x. Verify this empirically: fit `LogisticRegression` on the same two features and overlay its decision boundary on top of the linear-SVM boundary from part (b)(i). Are they both straight lines? Do they coincide exactly? If not, briefly explain why they can differ (*hint:* they optimise different objectives) even though both are linear.

(ii) **Kernelized prediction vanishes far from data. [4 pts]** In Problem 2(4) of this homework you argued that an RBF-kernel classifier of the form $h(x) = \sum_n \alpha_n k(x_n, x)$ goes to 0 when x is far from all training points. Verify this numerically. Using the RBF SVM at $\gamma = 1$ from part (d), pick a test point x^* far outside the training support (say, $(10, 10)$ in standardized coordinates—this is many standard deviations from any training point). Compute

$$h(x^*) = \sum_{n \in \mathrm{SV}} \alpha_n y_n \, k(x_n, x^*) + b,$$

where $\alpha_n y_n$ are available as `svc.dual_coef_` and x_n as `svc.support_vectors_`. Report $h(x^*)$. Is the magnitude close to $|b|$ (the intercept)? In 1–2 sentences, explain what this means for the confidence of the classifier on points that are very far from anything it has seen.

Problem 5 (Implementing SVMs from Scratch: Hinge vs. Logistic Loss)

It is a Tuesday night at a radio observatory. The dish has been pointed at a patch of sky for six hours; the pipeline has just finished folding the raw voltage stream into thousands of candidate signals. Most of these are radio-frequency interference, thermal noise, or satellites. A few—maybe one in ten—are real pulsars: rapidly rotating neutron stars that emit a narrow beam of radio emission once per rotation. The observatory does not have the budget to have a human astronomer stare at every candidate. Your job: build a classifier that triages candidates in real time so that the astronomer's attention lands on the real pulsars.

This problem has two goals. First, it is a from-scratch SVM implementation in NumPy—you will derive the gradients you already computed on paper in Problem 1, implement both the hinge-loss and the logistic-loss SVMs via (sub)gradient descent, and see them train. Second, it turns the theoretical claims of Problem 1 into empirical measurements: we will *count* the zero-gradient examples predicted by Problem 1(4), *plot* the margin-vs-gradient relationship predicted by Problem 1(3), and *stress-test* the outlier-sensitivity claim of Problem 1(5) by injecting label noise.

You may use `pandas` for loading / exploring the data and `matplotlib`/`seaborn` for plotting; **all model fitting and gradient computation must use NumPy**. In particular, do not use `sklearn.svm`.

Dataset. Download the *Predicting a Pulsar Star* dataset (HTRU2) from Kaggle:

https://www.kaggle.com/datasets/colearninglounge/predicting-pulsar-starintermediate

Each of the 17,898 rows is one candidate signal, with 8 continuous features extracted from the *integrated pulse profile* (what the folded waveform looks like) and the *DM-SNR curve* (how signal-to-noise varies with trial dispersion measure). The binary target is 1 if the candidate is a real pulsar and 0 otherwise.

Feature	Name	Description
x_1	`Mean_Integrated`	Mean of the integrated pulse profile
x_2	`SD_Integrated`	Standard deviation of the integrated pulse profile
x_3	`EK_Integrated`	Excess kurtosis of the integrated pulse profile
x_4	`Skewness_Integrated`	Skewness of the integrated pulse profile
x_5	`Mean_DMSNR`	Mean of the DM-SNR curve
x_6	`SD_DMSNR`	Standard deviation of the DM-SNR curve
x_7	`EK_DMSNR`	Excess kurtosis of the DM-SNR curve
x_8	`Skewness_DMSNR`	Skewness of the DM-SNR curve
y	`target_class`	1 if pulsar, 0 otherwise

(a) **EDA and preprocessing.**

(i) **Shape and class balance. [3 pts]** Load the CSV, print its shape and `.describe()`, and report the fraction of rows with `target_class` $= 1$. Why is *accuracy alone* going to be a misleading metric on this dataset?
Hint: Consider the trivial "always predict 0" classifier. What accuracy does it achieve here?

(ii) **Feature distributions by class. [3 pts]** Plot histograms of all 8 features in a 2×4 subplot grid. For each subplot, overlay two histograms—one for pulsars and one for non-pulsars—using different colours with some transparency (`alpha=0.5`). Which features visibly separate the two classes? Relate this to the margin intuition from Problem 1(3): features that separate the classes well are the ones that produce large $|m_n| = |y_n(w^T x_n + b)|$ once the model is trained.

(iii) **Preprocessing. [2 pts]** Convert labels to $y \in \{-1, +1\}$ (the SVM convention used throughout Problem 1 of this homework—not the $\{0, 1\}$ convention from logistic regression in Homework 1). Split the data into training, validation, and test sets using a 60/20/20 ratio with `np.random.seed(42)` (see Problem 5(b)(v) of Homework 1 for the rationale for three splits). Standardize all 8 features using **training-set** mean and standard deviation only. Append a column of ones to each feature matrix so that the bias term b can be folded into w, or keep it separate—either convention is fine, just be explicit about your choice.

(b) **Hinge-loss SVM via subgradient descent.**

Recall: The **subgradient** is a generalisation of the gradient to non-differentiable convex functions. The hinge loss $\ell(m) = \max(0, 1 - m)$ is non-differentiable at $m = 1$, but it is convex, and at every point (including the kink) it admits a valid subgradient. At $m < 1$

the subgradient is -1; at $m > 1$ it is 0; at $m = 1$ any value in $[-1, 0]$ is valid—we will just use 0. Subgradient descent uses a subgradient wherever the gradient is undefined.

(i) **Write down the gradient. [2 pts]** Quote your answer from Problem 1(1)–(2) of this homework: for one example (x, y) with margin $m = y(w^T x + b)$, the subgradient of ℓ_{hinge} with respect to w is

$$\partial_w \ell_{\text{hinge}} = -y\, x \cdot \mathbf{1}[m < 1], \qquad \partial_b \ell_{\text{hinge}} = -y \cdot \mathbf{1}[m < 1].$$

Write out the corresponding subgradient of the full objective $J_{\text{hinge}}(w, b) = \sum_n \max(0, 1 - m_n) + \frac{\lambda}{2}\|w\|^2$. (The $\frac{\lambda}{2}$ convention is cosmetic—it just makes the regulariser's gradient equal λw rather than $2\lambda w$.)

(ii) **Implement and train. [6 pts]** Implement full-batch subgradient descent for J_{hinge} with $\lambda = 10^{-3}$, learning rate $\eta = 10^{-3}$, 2000 iterations, and initialization $w^{(0)} = 0, b^{(0)} = 0$. Plot the training loss as a function of iteration. Does the loss decrease monotonically? (Subgradient descent is not guaranteed to—briefly explain why in 1 sentence if it does not.)

(iii) **Who is actually contributing? [4 pts]** Report training and validation accuracy. Then, at the final (w, b), count the number of training examples that satisfy $m_n \geq 1$—these are precisely the examples that contribute *zero* subgradient (Problem 1(4a)). Report the total count and the breakdown by class. On this imbalanced dataset, does most of the "active" mass (the examples still pulling on the boundary) sit on the rare (pulsar) class or the common (non-pulsar) class? Does that match your intuition?

(c) **Logistic-loss SVM.**

(i) **Write down the gradient. [2 pts]** From Problem 1(1)–(2), the gradient of $\ell_{\log}(w, b) = \log(1 + e^{-m})$ with respect to w is

$$\nabla_w \ell_{\log} = -\sigma(-m)\, y\, x, \qquad \frac{\partial \ell_{\log}}{\partial b} = -\sigma(-m)\, y,$$

where $\sigma(z) = 1/(1 + e^{-z})$. Write out the gradient of the full $J_{\log}(w, b) = \sum_n \log(1 + e^{-m_n}) + \frac{\lambda}{2}\|w\|^2$.

(ii) **Implement and train. [6 pts]** Implement full-batch gradient descent for $J_{\log}$ with the same $\lambda = 10^{-3}$, $\eta = 10^{-3}$, 2000 iterations, and zero initialization. Plot the loss curve on the *same axes* as the hinge-loss curve from (b)(ii), using a log-scaled y-axis if the two curves live on very different scales. Which loss converges faster at this step size? Briefly explain in terms of the shape of the two losses.
Numerical note: Implement $\log(1 + e^{-m})$ using `np.logaddexp(0, -m)`—the naive `np.log(1+np.exp(-m))` will overflow when m is very negative, producing `inf`.

(iii) **Is any example ever exactly inactive? [4 pts]** Report training and validation accuracy. As a direct empirical check of Problem 1(4b), count:

- how many training examples have $\|\nabla_w \ell_n\|_2$ *exactly* zero at the final (w, b),
- how many have it below 10^{-6}.

Contrast these counts with the count from (b)(iii). Does this match what Problem 1(4b) predicted—namely, that logistic loss never assigns exactly zero gradient to an example, even a very well-classified one?

(d) **Margin vs. gradient magnitude—empirical check of Problem 1(3). [10 pts]**

Using the fitted *logistic* model from (c)(ii), for every training example n compute its margin m_n and its per-example gradient norm $\|\nabla_w \ell_n\|_2 = \sigma(-m_n) \cdot \|x_n\|_2$. Produce a scatter plot with m_n on the horizontal axis and the gradient norm on the vertical axis. Verify all three qualitative regimes from Problem 1(3):

- $m \gg 0$: gradient norm $\to 0$ exponentially. Easy, confidently correctly-classified points stop mattering to the update.
- $m \approx 0$: gradient norm $\approx \frac{1}{2}\|x\|_2$. Points right on the decision boundary contribute the most per-example gradient (modulo their own feature-norm).
- $m \ll 0$: gradient norm saturates at $\|x\|_2$. Confidently *mis*classified points are bounded, not exploding.

Now, on the same axes, overlay the hinge-loss per-example (sub)gradient norm $\|x_n\|_2 \cdot \mathbf{1}[m_n < 1]$—a hard step from $\|x_n\|_2$ down to 0 at $m = 1$. In 2–3 sentences, explain the visual contrast that Problem 1(5) predicted: hinge is a binary "you matter / you don't" switch at $m = 1$, while logistic is a smooth sigmoidal interpolation in m that never turns off entirely.

(e) **Robustness to label noise. [8 pts]**

So far we have trained on clean labels. In the real observatory, catalogues contain mislabelled candidates—e.g. an RFI spike that was accidentally tagged as a pulsar. A good classifier should not be dragged around too much by a handful of wrong labels.

Corrupt 5% of the *training* labels by flipping them uniformly at random (seed the RNG with, e.g., `np.random.seed(0)` so your experiment is reproducible). Do **not** touch the validation or test labels. Refit both models from scratch with the same hyperparameters as in (b)(ii) and (c)(ii). Report:

(i) the change in validation accuracy for each model (clean vs. noisy training),

(ii) the change in $\|w\|_2$ for each model.

Which loss is more sensitive to label noise, and does this match the intuition you gave in Problem 1(5)?
Expected direction, not the final answer: for a confidently-wrong point ($m \ll 0$), the hinge subgradient is $-yx$ with magnitude $\|x\|$ that scales linearly in how wrong you are—there is no bound on its contribution per step. The logistic gradient is $-\sigma(-m)yx$ with magnitude bounded by $\|x\|$ regardless of how wrong m is. Predict based on this which loss should be dragged further by flipped labels, then check.

(f) **Regularization sweep and final evaluation.**

Return to clean labels. For the logistic-loss objective only, sweep

$$\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1, 10\}.$$

(i) **[5 pts]** Retrain from zero initialisation at each λ (same η and iteration budget as in (c)(ii)) and report training and validation accuracy.

(ii) **[3 pts]** Plot training and validation accuracy as a function of $\log_{10} \lambda$ on the same axes. Pick the λ^* that maximises **validation** accuracy.
Why validation, not test: identical reasoning to Problem 5(f)(iii) of Homework 1—the test set is reserved for a single honest evaluation at the very end.

(iii) **[2 pts]** Retrain at λ^* on the concatenated train+validation data and report the final **test** accuracy.

(g) **Summary and discussion. [5 pts]**

Produce a single table summarising all models you trained: the hinge-loss model from (b), the logistic-loss model from (c), the two label-noise variants from (e), and the best λ^* model from (f). Columns: training accuracy, validation accuracy, test accuracy (when available), $\|w\|_2$. In 3–5 sentences, discuss:

- Which loss would you ship to the observatory and why?
- Did the empirical results in parts (d) and (e) confirm the theoretical predictions from Problem 1(3) and Problem 1(5)? If any did not match, speculate briefly about why.
- Name one thing you would try next. (For example: kernelize the model with an RBF kernel—as in Problem 4—or use stochastic rather than full-batch gradient steps.)